- Основные концепции numpy
- Типизация, .dtype и .astype
- Векторизованные операции и агрегации
- Понятие оси и работа с axis
- Булевы маски и фильтрация

Программирование в лингвистике

Лекция 13

NumPy: базовый уровень

Зачем numpy в Python?

- Список в Python (list) хранит указатели на разные объекты, которые лежат в разных частях памяти. Он гибкий (можно хранить строки, числа, объекты вместе), но объемный и медленный при вычислениях из-за накладных расходов на типы данных.
- В numpy основное понятие и тип данных – это массив (np.array). Он хранит данные одного типа подряд в непрерывном участке памяти (буфер фиксированной длины). За счет этого процессор обрабатывает их в разы быстрее, и тратится меньше памяти.
- Операции применяются ко всем элементам массива сразу, без написания циклов for на Python. Под капотом это работает на оптимизированном коде C.

```
import numpy as np

py_list = [1, 2, 3, 4]
# Обычный список

np_arr = np.array([1, 2, 3, 4])
# Массив NumPy

print(py_list == np_arr)
# Умеем сравнивать!
```

```
[True True True True]
```

Типы данных и параметр .dtype

- В связи с реализацией на C в массивах numpy данные типизируются похожим на C способом.
- Типы чисел более дробные: есть беззнаковые целые (unsigned integer) - у которых не хранится знак; есть int и float разных размеров, от 8 до 64 битов (аналоги short/long в C) и т.д.
- Все массивы в numpy явно типизированы. Тип массива хранится в параметре .dtype.
- Если мы ввели данные разных типов при создании массива, они гомогенизируются (upcasting).
- Например, все целые числа, введенные вперемешку с дробными станут при создании массива дробными. Если смешать числа со строками, все станет строками (unicode string).

```
import numpy as np

arr1 = np.array([1, 2, 3.5])
print(arr1)
print(arr1.dtype)

arr2 = np.array[1, 2, "hello"])
print(arr2)
print(arr2.dtype)
```

```
[1.  2.  3.5]
float64
['1' '2' 'hello']
<U21
```

Управление типом и метод .astype()

- Чтобы получить массив нужного нам типа, можно сразу указать его при создании.
- Но бывает, что нужно поменять тип уже созданного массива.
- В этом случае используем метод .astype()
- **Важно:** на самом деле он не меняет старый массив, а возвращает новый.
- Менять тип нужно для операций.

Для операций с небольшими числами:

int8, int16, float16, float32

Для операций с большими числами:

int32, int64, float64, float128

Для строк:

str_, bytes_

```
import numpy as np

arr1 = np.array([1, 2, 3], dtype=np.float64)
print(arr1)
arr2 = arr1.astype(np.int64)
print(arr2)
```

```
[1. 2. 3.]
[1 2 3]
```

Типы данных и экономия памяти

- Оптимизация типа данных важна для экономии памяти.
- Посмотрим, сколько памяти занимает массив из маленьких чисел (например, возраст людей), если хранить его по умолчанию и если оптимизировать тип данных.
- Для этого используем атрибут .nbytes (показывает размер массива в байтах).

Путем оптимизации типа мы, не потеряв ни единого бита полезной информации, использовали памяти в 8 раз меньше!

```
import numpy as np

# Создаем массив «возраст сотрудников» (1 млн записей), по умолчанию NumPy создаст int64
ages_default = np.random.randint(18, 65, size=1000000)
# Оптимизируем: возраст легко помещается в int8
ages_optimized = ages_default.astype(np.int8)

# Считаем вес в мегабайтах
mb_default = ages_default.nbytes / (1024 ** 2)
mb_optimized = ages_optimized.nbytes / (1024 ** 2)

print(f"Размер в int64 (по умолчанию): {mb_default:.2f} МБ")  # ~7.63 МБ
print(f"Размер в int8 (оптимизированный): {mb_optimized:.2f} МБ")  # ~0.95 МБ
```

Что такое векторизация?

- Векторизация – это отказ от циклов на уровне Python в пользу одновременной обработки всех элементов массива на уровне быстрого C-кода.
- В NumPy мы просто пишем математическое выражение для всего массива сразу.
- **Почему это работает мгновенно?** NumPy заранее знает, что внутри массива лежат, например, только int64. Ему не нужно проверять тип каждого элемента.
- Массив передается как непрерывный блок памяти процессору, и тот одной командой обрабатывает всю пачку чисел.

```
import numpy as np

prices_np = np.array([100, 200, 300])
new_prices_np = prices_np + 10
# Векторизованная операция!

print(new_prices_np)
```

```
[110 210 310]
```

Попробуйте сравнить с циклом по времени!

Четыре сценария векторизации

- Массив и скаляр (число): операция применяется к каждому элементу массива.
- Массив и массив (одинаковой длины): операция производится поэлементно (по индексам).
- Векторизованные математические функции (np.sin, np.log, np.exp): модуль math из Python не умеет работать с массивами. В NumPy для этого есть свои встроенные функции.
- Векторизованные сравнения: результат - массив из True и False (элементы bool_ типа).

* bool в NumPy – не подтип int, как в Python, но подтип общего типа чисел number_, размер его – строго 8 бит (1 байт).

```
import numpy as np

nums = np.array([500, -10, 1500, 0])
is_negative = nums < 0
# Векторизованное сравнение

print(is_negative)
print(is_negative.dtype)
```

```
[False  True False False]
bool
```

Агрегации для одномерных массивов

Агрегации в NumPy – это операции, которые сводят множество чисел из массива в одно или несколько итоговых значений (например, сумму, максимум или среднее).

Метод	Пример	Результат	Пояснение
np.sum	np.sum([1, 2, 3, 4, 5])	15	Считает общую сумму элементов.
np.mean	np.mean([1, 2, 3, 4, 5])	3.0	Находит среднее арифметическое.
np.min np.max	np.min([1, 2, 3, 4, 5]) np.max([1, 2, 3, 4, 5])	1 5	Находят минимальное или максимальное значение.
np.std	np.std([1, 2, 3, 4, 5])	1.41421356	Вычисляет стандартное отклонение (разброс данных).
np.argmin np.argmax	np.argmax([1, 2, 3, 4, 5]) np.argmax([1, 2, 3, 4, 5])	0 4	Возвращают индекс минимального или максимального элемента.

Агрегация по осям

Агрегации для двумерных массивов работают с осями (axis). Параметр оси показывает NumPy, по какому направлению схлопывать данные.

axis=0 – агрегация по столбцам (результат для каждого столбца).

axis=1 – агрегация по строкам (результат для каждой строки).

Параметр keepdims=True позволяет сохранить исходную размерность массива, не превращая его в плоский вектор, что критически важно для корректного вычитания или деления матриц.

```
import numpy as np

matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

print("Общая сумма:", np.sum(matrix)) # Результат: 45
print("Сумма по столбцам:", np.sum(matrix, axis=0))
# Результат: [12 15 18]
print("Максимум по строкам:", np.max(matrix, axis=1))
# Результат: [3 6 9]

res_flat = np.sum(matrix, axis=1)
print("Без keepdims:", res_flat, "Форма:", res_flat.shape)
# Результат: [6 15 24] Форма: (3,)
res_dims = np.sum(matrix, axis=1, keepdims=True)
print("С keepdims:\n", res_dims, "Форма:", res_dims.shape)
# Результат: [[6] [15] [24]] Форма: (3, 1)
```

Специальные агрегации

Обычные функции агрегации вернут nan, если в массиве есть хотя бы одно неопределенное значение. Для корректного подсчета по оставшимся числам используются специальные nan-версии функций:

- nansum для безопасной суммы;
- nanmean для безопасного среднего;
- nanstd для стандартного отклонения, и т.д.

Логические агрегации спользуются для проверки условий на массивах с булевыми значениями:

- np.any – хотя бы один элемент равен True?
- np.all – все элементы равны True?

```
import numpy as np

bool_arr = np.array([True, False, True])

#Any
print(np.any(bool_arr))
#All
print(np.all(bool_arr))
```

```
True
False
```

Булевы маски

- Булева маска – это массив, который создается на основе другого массива. Он состоит только из элементов типа bool, соответствующих элементам исходного массива и отражающих то, подходят ли эти исходные элементы под некоторое заданное условие.
- Булева маска всегда имеет точно такой же размер, как исходный массив.
- Булеву маску можно использовать для фильтрации данных, если передать ее внутрь квадратных скобок исходного массива.

```
import numpy as np

balances = np.array([1500, -200, 0, 450, -15])
debtor_mask = balances < 0
print(debtor_mask)
#Фильтруем данные по маске
bad_balances = balances[debtor_mask]
print(bad_balances)
```

```
[False  True False False  True]
[-200  -15]
```

Логические операторы применяются к булевым маскам поэлементно и похожи на операторы из C: вместо and пишем &, вместо or пишем | и вместо not пишем ~

СПАСИБО ЗА ВНИМАНИЕ!

Контакты для связи: ajumo@yandex.ru, abyssian03@gmail.com

GitHub: https://github.com/abyssian03/Stud-2